

Informe Laboratorio 4

Sección 1

Benjamín Zúñiga Pueller
e-mail: benjamin.zuniga2@mail.udp.cl

Mayo de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Investiga y documenta los tamaños de clave e IV	3
2.2. Solicita datos de entrada desde la terminal	3
2.3. Valida y ajusta la clave según el algoritmo	4
2.4. Implementa el cifrado y descifrado en modo CBC	5
2.5. Compara los resultados con un servicio de cifrado online	6
2.6. Describe la aplicabilidad del cifrado simétrico en la vida real	8

1. Descripción de actividades

Desarrollar un programa en Python utilizando la librería `pycryptodome` para cifrar y descifrar mensajes con los algoritmos DES, AES-256 y 3DES, permitiendo la entrada de la key, vector de inicialización y el texto a cifrar desde la terminal.

Instrucciones:

1. Investigación

- Investigue y documente el tamaño en bytes de la clave y el vector de inicialización (IV) requeridos para los algoritmos DES, AES-256 y 3DES. Mencione las principales diferencias entre cada algoritmo, sea breve.

2. El programa debe solicitar al usuario los siguientes datos desde la terminal

- Key correspondiente a cada algoritmo.
- Vector de Inicialización (IV) para cada algoritmo.
- Texto a cifrar.

3. Validación y ajuste de la clave

- Si la clave ingresada es menor que el tamaño necesario para el algoritmo complete los bytes faltantes agregando bytes adicionales generados de manera aleatoria (utiliza `get_random_bytes`).
- Si la clave ingresada es mayor que el tamaño requerido, trunque la clave a la longitud necesaria.
- Imprima la clave final utilizada para cada algoritmo después de los ajustes.

4. Cifrado y Descifrado

- Implemente una función para cada algoritmo de cifrado y descifrado (DES, AES-256, y 3DES). Use el modo CBC para todos los algoritmos.
- Asegúrese de utilizar el IV proporcionado por el usuario para el proceso de cifrado y descifrado.
- Imprima tanto el texto cifrado como el texto descifrado.

5. Comparación con un servicio de cifrado online

- Selecciona uno de los tres algoritmos (DES, AES-256 o 3DES), ingrese el mismo texto, key y vector de inicialización en una página web de cifrado online.
- Compare los resultados de tu programa con los del servicio online. Valide si el resultado es el mismo y fundamente su respuesta.

6. Aplicabilidad en la vida real

- Describa un caso, situación o problema donde usaría cifrado simétrico. Defina que algoritmo de cifrado simétrico recomendaría justificando su respuesta.
- Suponga que la recomendación que usted entrego no fue bien percibida por su contraparte y le pide implementar hashes en vez de cifrado simétrico. Argumente cuál sería su respuesta frente a dicha solicitud.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Investiga y documenta los tamaños de clave e IV

El tamaño del vector de inicialización y el tamaño de bytes para los algoritmos DES, 3DES y AES-256 se pueden ver en la Tabla 1:

Algoritmo	Vector de Inicialización	Clave
DES	8 bytes	8 bytes
3DES	8 bytes	16 bytes o 24 bytes
AES-256	16 bytes	32 bytes

Tabla 1: Valor en bytes de los vectores de inicialización y claves de los algoritmos.

DES (Data Encryption Standard): cifra bloques de 64 bits utilizando una clave de 56 bits efectivos. Opera mediante permutaciones y sustituciones en 16 rondas. Hoy en día se considera obsoleto e inseguro debido a su corta longitud de clave, la cual puede ser vulnerada mediante ataques de fuerza bruta.

3DES (Triple DES): aplica el algoritmo DES tres veces consecutivas sobre cada bloque. Los bloques se mantienen en 64 bits, pero se utilizan dos o tres claves distintas de 8 bytes cada una, alcanzando una clave efectiva de 112 a 168 bits según la variante. Si bien mejora la seguridad respecto a DES, también está considerado obsoleto frente a alternativas modernas.

AES-256 (Advanced Encryption Standard): es el estándar de cifrado simétrico vigente. Divide la información en bloques de 16 bytes, organizados en una matriz 4×4 , y aplica una serie de transformaciones matemáticas (sustitución, permutación, mezcla de columnas y adición de clave de ronda) a lo largo de 14 rondas determinadas por su clave de 256 bits. Es considerado seguro para uso en producción y es el algoritmo recomendado actualmente.

2.2. Solicita datos de entrada desde la terminal

El programa solicita al usuario, de forma interactiva en la terminal, los tres datos requeridos para operar: la clave (*key*), el vector de inicialización (IV) y el texto a cifrar.

En la Figura 1 se puede observar cómo el programa presenta este flujo de ingreso de datos en la terminal.

```
Ingresa la clave (key) : Universidad Diego Portales
Ingresa el IV          : UDP
Ingresa el texto       : Hola como estas
```

Figura 1: Solicitud de datos vía terminal: selección de algoritmo e ingreso de clave, IV y texto.

2.3. Valida y ajusta la clave según el algoritmo

Una vez ingresados los datos, el programa valida que tanto la clave como el IV tengan el tamaño en bytes requerido por el algoritmo seleccionado. Para ello se aplica la siguiente lógica:

- Si la clave o el IV son **más cortos** que el tamaño requerido, se completan los bytes faltantes con bytes aleatorios generados mediante la función `get_random_bytes` de la librería `pycryptodome`.
- Si la clave o el IV son **más largos** que el tamaño máximo permitido, se truncan hasta el tamaño necesario.
- En ambos casos, el programa imprime en pantalla la clave final resultante en formato hexadecimal y en Base64, permitiendo al usuario verificar exactamente qué clave fue utilizada en el proceso de cifrado.

La Figura 2 muestra la salida del programa durante esta etapa, donde se aprecia el ajuste realizado sobre la clave e IV ingresados.

```

[AES] key válida: [16, 24, 32]B | IV requerido: 16B
— Clave —
Tamaño recibido : 26B → objetivo: 32B
Acción          : COMPLETADO (+6B aleatorios)
Clave final(hex): 556e69766572736964616420446965676f20506f7274616c657371f108806e9e
Clave final(b64): VW5pdmVyc2lkYWQgRGllZ28gUG9ydGFsZXNx8Q1Abp4=
— IV —
Tamaño recibido : 3B → requerido: 16B
Acción          : COMPLETADO (+13B aleatorios)
IV final(hex)   : 554450a14216a360fbbd10e384715162

[DES] key válida: [8]B | IV requerido: 8B
— Clave —
Tamaño recibido : 26B → objetivo: 8B
Acción          : TRUNCADO a 8B
Clave final(hex): 556e697665727369
Clave final(b64): VW5pdmVyc2k=
— IV —
Tamaño recibido : 3B → requerido: 8B
Acción          : COMPLETADO (+5B aleatorios)
IV final(hex)   : 5544505162700e00

[3DES] key válida: [16, 24]B | IV requerido: 8B
— Clave —
Tamaño recibido : 26B → objetivo: 24B
Acción          : TRUNCADO a 24B
Clave final(hex): 556e69766572736964616420446965676f20506f7274616c
Clave final(b64): VW5pdmVyc2lkYWQgRGllZ28gUG9ydGFs
— IV —
Tamaño recibido : 3B → requerido: 8B
Acción          : COMPLETADO (+5B aleatorios)
IV final(hex)   : 554450a3a97b45fb

```

Figura 2: Validación y rectificación de la clave y del IV según el algoritmo seleccionado.

2.4. Implementa el cifrado y descifrado en modo CBC

El programa implementa funciones independientes de cifrado y descifrado para cada uno de los tres algoritmos: `cifrar_aes / descifrar_aes`, `cifrar_des / descifrar_des` y `cifrar_3des / descifrar_3des`. Todos operan en modo CBC (*Cipher Block Chaining*), el cual requiere un IV para encadenar los bloques y añade una capa adicional de seguridad respecto al modo ECB, ya que bloques de texto plano idénticos producen bloques cifrados distintos.

Para garantizar que el texto sea un múltiplo del tamaño de bloque requerido por cada algoritmo, se aplica un relleno PKCS7 (*padding*) antes del cifrado, el cual es removido después del descifrado.

El proceso completo es el siguiente:

1. Se aplica padding PKCS7 al texto original.
2. Se cifra utilizando la clave y el IV validados, en modo CBC.
3. El resultado cifrado se muestra en formato hexadecimal y Base64.
4. Se descifra el texto cifrado utilizando la misma clave e IV.
5. Se remueve el padding y se imprime el texto descifrado, que debe coincidir con el texto original.

En la Figura 3 se observa la salida del programa con el texto original, el texto cifrado y el texto descifrado resultante, confirmando que el proceso es reversible y correcto.

```

Texto original : Hola como estas

AES
Cifrado (hex) : a50ba7220f9016dc7518c483752cf0a5b1a330b9e7048b27976f8f63d45e8815
Cifrado (b64) : pQunIg+QFtx1GMSDdSzwpbGjMLnnBIsnI2+PY9ReiBU=
Descifrado   : Hola como estas
✓ Coincide con el original

DES
Cifrado (hex) : f6594019fe8c2286b5f644982941f3916f179d6c30cf6ddb
Cifrado (b64) : 9lLAGf6MIoa19kSYKUHkzW8XnWwz23b
Descifrado   : Hola como estas
✓ Coincide con el original

3DES
Cifrado (hex) : 639a04d950aa9079e8334885877768ea1f536c17a693802f
Cifrado (b64) : YSoE2VCqkHnoM0iFh3do6h9TbBemk4Av
Descifrado   : Hola como estas
✓ Coincide con el original

```

Figura 3: Resultado del proceso de cifrado y descifrado con el algoritmo seleccionado en modo CBC.

2.5. Compara los resultados con un servicio de cifrado online

Una vez ya hecho todo lo anterior se procedió a comparar los resultados utilizando un encriptador online. Donde se puede apreciar tanto en la Figura 4 Y 5 como se ingresa el bloque que se quiere encriptar que en este caso fue “Hola como estas” ademas con las keys e IV dada para cada uno de los algoritmos, finalmente se puede observar que el bloque que se nos entrega en el output es el mismo que nos entrego nuestro programa hecho en python.

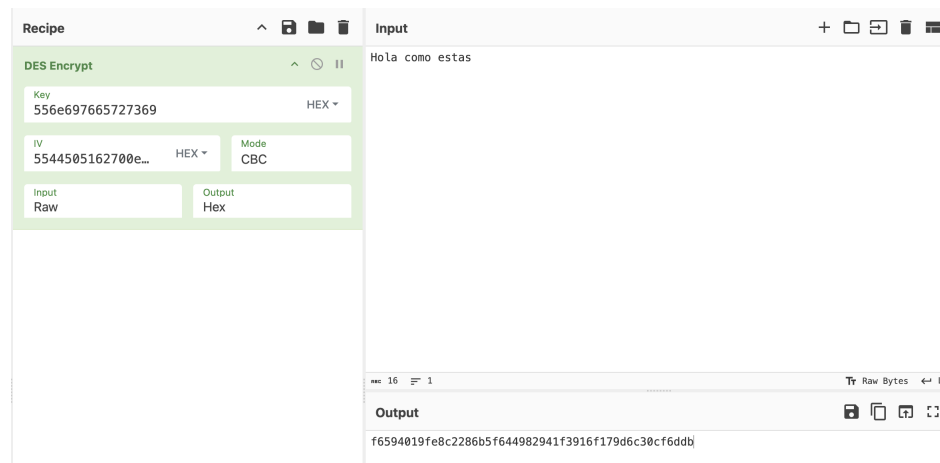


Figura 4: Bloque encriptado por servicio de cifrado online usando DES

2 DESARROLLO DE ACTIVIDADES SEGÚN CRITERIO DE RÚBRICA

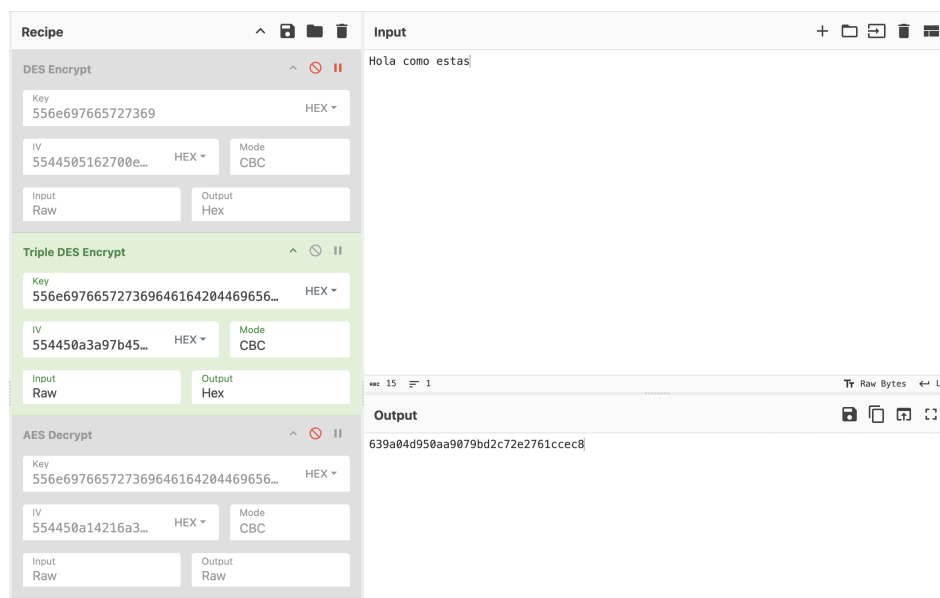


Figura 5: Bloque encryptado por servicio de cifrado online usando 3DES

También se hizo lo inverso con AES donde se le entrega el bloque ya codificado para que lo des-encrypte. En la Figura 6 se ve como al momento de ingresar el bloque cifrado que hizo el programa en Python aparece el bloque “Hola como estas” en el output indicando que nuestro script puede cifrar correctamente los bloques que se le entregan.

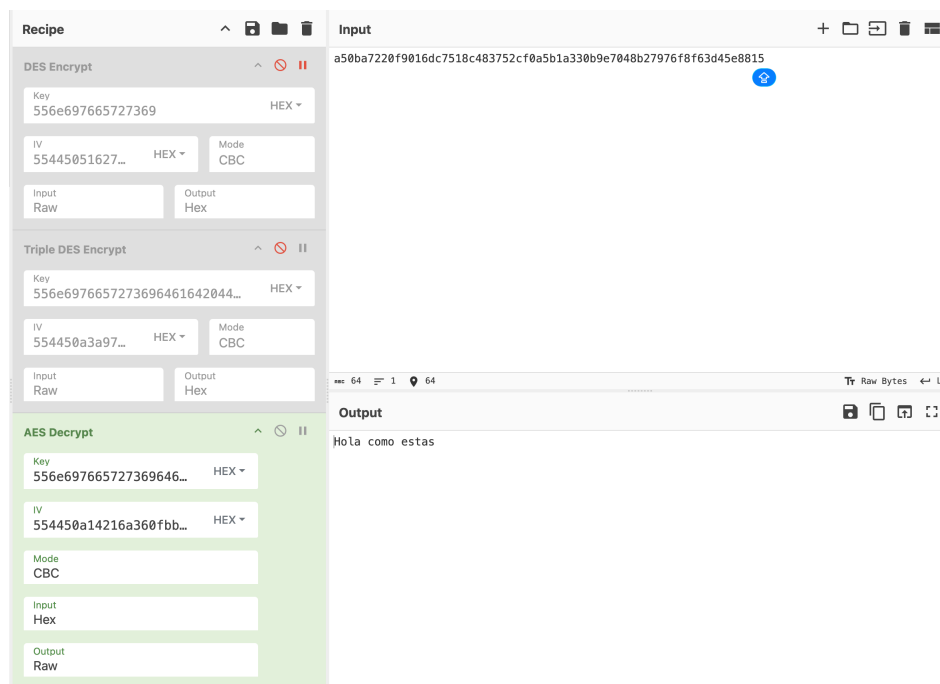


Figura 6: Desencryptado realizado por el servicio de cifrado online

2.6. Describe la aplicabilidad del cifrado simétrico en la vida real

Caso de uso propuesto

Un caso concreto donde se aplicaría cifrado simétrico es el **almacenamiento de archivos sensibles en un sistema de gestión documental interno** de una empresa. Por ejemplo, una clínica médica que almacena fichas de pacientes en un servidor local necesita garantizar que, incluso si el servidor es comprometido físicamente, los archivos no sean legibles por terceros. En este escenario, los archivos se cifran antes de ser almacenados y se descifran solo cuando un usuario autenticado los solicita.

Algoritmo recomendado

Se recomienda utilizar **AES-256 en modo CBC o GCM**. AES-256 es el estándar vigente aprobado por el NIST, ofrece una seguridad robusta con una clave de 256 bits, y tiene implementaciones altamente eficientes en hardware y software modernos. El modo GCM (*Galois/Counter Mode*) es especialmente recomendado en entornos productivos por añadir autenticación integrada, detectando modificaciones no autorizadas del texto cifrado.

Respuesta ante la solicitud de usar hashes en lugar de cifrado simétrico

La solicitud de reemplazar el cifrado simétrico por funciones hash no es apropiada para este caso de uso, ya que responde a una confusión conceptual entre dos herramientas criptográficas con propósitos distintos.

Un **hash** es una función de sentido único: transforma una entrada en una cadena de longitud fija, pero **no es reversible**. Su utilidad radica en la verificación de integridad (por ejemplo, comprobar si una contraseña ingresada coincide con su hash almacenado), no en la recuperación del dato original.

El **cifrado simétrico**, en cambio, es reversible: permite recuperar el mensaje original a partir del texto cifrado, siempre que se disponga de la clave correcta. Esto es precisamente lo que se necesita al almacenar y recuperar documentos completos.

Por lo tanto, si el objetivo es proteger archivos que luego deben ser accesibles y leídos por usuarios autorizados, el cifrado simétrico es la herramienta correcta. Utilizar hashes en su lugar haría imposible recuperar los documentos originales, inutilizando completamente el sistema.

Conclusiones y comentarios

A lo largo de este laboratorio se implementó exitosamente un programa en Python capaz de cifrar y descifrar texto mediante los algoritmos DES, 3DES y AES-256, operando en modo CBC con claves e IV provistos por el usuario.

Se comprobó que la gestión correcta del tamaño de la clave y el IV es fundamental: una clave de longitud incorrecta impide que el algoritmo opere, por lo que el ajuste automático mediante bytes aleatorios o truncamiento resulta una solución práctica. Sin embargo, en un

entorno productivo real, lo recomendable sería exigir al usuario una clave del tamaño exacto para evitar pérdida de entropía.

Asimismo, la comparación con un servicio online permitió validar que la implementación produce resultados estándar e interoperables, siempre que todos los parámetros (modo, padding, codificación) sean consistentes.

Finalmente, queda clara la diferencia conceptual entre cifrado y hashing: cada uno tiene su lugar en la criptografía aplicada y no son intercambiables. AES-256 se posiciona como la opción recomendada para proteger información confidencial en aplicaciones modernas, dado su nivel de seguridad, eficiencia y amplio soporte en la industria.